

Value-driven Regression Test Selection

Abstract

This paper introduces a value-driven approach to regression test selection to enhance the effectiveness of regression testing in agile development. By selecting test cases associated with higher business value, this approach ensures that critical existing business functionalities are thoroughly re-tested in case of any changes during the testing process. Unlike traditional approaches focusing on technical coverage, this method integrates business priorities, resulting in a more targeted and strategic test selection. Simulations comparing this approach to random selection demonstrate that it preserves more business value while reducing the number of test cases executed. The results indicate that this value-driven regression testing selection method strikes an optimal balance between testing efficiency and safeguarding high-impact features, making it a practical and effective solution for agile environments.

Keywords

Software Engineering, Software Testing, Verification, Validation, Agile, Regression Testing, Regression Test Selection

ACM Reference Format:

. 2025. Value-driven Regression Test Selection. In *Proceedings of ACM SAC Conference (AlxSE'25)*. ACM, New York, NY, USA, Article 4, 8 pages. https://doi.org/xx.xxx/xxx_x

1 INTRODUCTION

This paper introduces a Regression Test Selection (RTS) approach tailored for agile methods. Agile methods are the dominant software development method, with over 89% [16] organizations following some agile variants. The quick nature of agile comes with challenges, and testing is one of the significant ones [9]. The challenge becomes more evident after an update/change to ensure delivered functionalities continue to work as intended, suggesting that the software has not regressed backward.

Regression testing is the process that aims to ensure that software changes do not break existing accepted functionalities [19]. The swift deliveries promised by agile methodologies and CI/CD pipelines can compromise quality, resulting in businesses losing money and face. The CrowdStrike incident in 2024 summer is a stark reminder of the potential risks of incomplete regression testing, demonstrating how it can lead to critical failures [7]. This incident underscores the need for thorough testing, as incomplete regression testing under the pressure of fast delivery [8] can lead to reliance on automated tests set up by engineers who may lack a

comprehensive understanding of the system's functionality from a business perspective that is valuable to the customer.

Agile, as per the manifesto [2], strongly emphasizes the 'value' of the software product delivered, which is the primary success metric of the deliverable. This value, as highlighted by Hidenberg et al. [2012] [11], is not just financial but also non-financial. Moreover, according to Poppendieck [2011] [18], lean principles, any activity that does not provide value to the customer/organization should be eliminated. We extend this concept to regression testing, which, while not directly contributing value, plays a crucial role in checking the amount of value the product holds after changes. It ensures that new increments do not introduce defects that could diminish the value of the already delivered software; thus, regression testing is a value-preserving activity.

Requirements hold value, and researchers have explored requirement based regression testing [6] [23]. Various techniques are used to rank requirements, followed by selecting and/or prioritizing test cases associated with the requirements for regression testing. While these approaches report prominent results, they have certain limitations regarding the evaluation metric. Even though promising, these metrics used to evaluate the effectiveness of regression testing have yet to capture the value of the requirement selected and/or prioritized from a business point of view. For example, the software updates a sorting algorithm in a component, and the component is thoroughly regression tested. However, the changes are irrelevant to the end user's business needs, i.e., the change does not affect the value of the software. However, for large enterprise systems such as CrowdStrike, we cannot say that this change will not regress the whole system even though the component might not have regressed. Thus, regression testing post any increment must include metrics that signify the business value of requirements. This metric must be used simultaneously with metrics such as coverage/fault detection rate to ensure that the change has not impacted anything technical or the business.

Hence, we suggest an approach in which a requirement's business value is considered when selecting test cases for regression testing. If a requirement already adds value to the system, it has fulfilled all the necessary acceptance criteria, addressed the business needs, and met the definition of done (DoD) [15]. Therefore, from a regression testing standpoint, the goal is to ensure that all requirements that meet the definition of done remain in the done state.

Given the role of regression testing in agile environments, it is essential to ensure technical and component-level functional correctness and safeguard the software's business value. While current regression testing approaches excel at testing technical changes, they may overlook the broader implications of how these changes impact the system as a whole. By integrating business value into the test selection process, regression testing can evolve into a more strategic tool that ensures the software's stability and continued alignment with business priorities. This refined approach opens the door to a deeper exploration of how value-based testing

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

AlxSE'25, Laguna Hills, CA

© 2025 ACM.

ACM ISBN 978-x-xxxx-xxxx-x/YYYY/MM

https://doi.org/xx.xxx/xxx_x

can enhance the effectiveness of regression testing in today's fast-paced development landscape.

The rest of this paper is organized as follows: section 2 lays the foundation for understanding how regression test selection strategies can be effectively applied in agile environments. Section 3 discusses the design rationale and implementation of our algorithm. Section 4 describes the experimental approach for validation of the algorithm, and section 5 present the results of that experimentation. Section 6 discusses our interpretation of the results, and we conclude with some final thoughts in section 8.

2 Background

A regression test selection (RTS) algorithm selects a subset of tests from the complete set of available tests, reducing the size of the regression test suite and improving testing efficiency [4]. Many different criteria have been presented in the literature for guiding RTS, such as cite Arafeen2013:

- selecting tests associated with requirements that have higher significance
- selecting tests that are more likely to detect defects (based on execution history)
- selecting tests closely related to new requirements introduced since the last version of the software
- selecting tests based on what code has changed since the last version

A related concept is regression test prioritization (RTP), which extends RTS by requiring an ordered subset, presumably with the motivation of finding (more and/or significant) defects earlier. For the purposes of this research, we consider the mathematically weaker notion of RTS instead of RTP, for reasons discussed in section 2.3. RTS and RTP algorithms have been studied for a long time, with established approaches identified by XXX, XXX, and XXX among others. However, agile software processes, with their emphasis on short (or continuous) release cycles of smaller software versions, suggest revisiting these algorithms and approaches to address the nature of what is now the predominant way to develop and deliver software.

The subsequent subsections will explore some of the significant contributions of requirement-based RTS, which selects and prioritizes test cases aligned with essential requirements; textual similarity-based RTS, which groups test cases based on the similarity of requirements, enabling efficient selection; and agile-specific regression testing techniques, which adapt regression testing practices to the rapid cycles of continuous integration and delivery (CI/CD) in agile methodologies.

2.1 Requirements-based Regression Test Selection

The approach we present in this paper is based on requirements significance (1st bullet above), and thus fits into the class of RTS algorithms based on requirements priority. Multiple studies [3], [6], [23] address requirements-based regression testing. One of the studies that motivated this paper is the technique proposed by Chittimalli et al. [2008]. The study provides a way to use system requirements and their associations with test cases to perform RTS. In addition, the proposed technique uses this criticality to order the

selected test cases. Another study by Srivastava et al. [2008] proposed a requirements-based regression test prioritization technique that explores two factors: 1) the priority of requirements given by customers, developers, and managers, and 2) the severity and probability of risk factors that occur in requirements. Khalid et al. [2019] [13] collected a dataset of black box test cases using a simple ranking method, and assigned user priority weights according to the priority assigned during the requirement elicitation phase.

2.2 Textual Similarity Based Regression Test Selection

For this paper, the studies on requirement clustering and measuring textual similarity between requirements closely align with our research and provide significant motivation. Notably, our work resembles the study by Arafeen et al. [2013] [3], which uses textual similarity to cluster requirements for test case prioritization. Their approach involves calculating textual resemblance between requirements and applying k-means clustering to group similar requirements. The clusters are then prioritized based on two factors: 1) code commits by developers and 2) the extent of code modifications. Test cases are selected from these prioritized clusters using a requirement-test case traceability matrix. The technique's effectiveness was validated using the Average Percentage of Fault Detection, showing improvement rates between 55.52% and 65.37%. However, the method's reliance on accessing source code, especially in large applications, presents challenges.

For our research, textual similarity is used as a means to relate new requirements to similar past requirements, and thus provide one dimension of our selection algorithm. This way of localizing the impact of changes in the requirements space (compared to the code space, 4th bullet above) is less intensive and does not even require the source code. To be clear, we are using an off-the-shelf (OTS) textual similarity library, not contributing a new such algorithm. From a test process standpoint, better textual similarity algorithms, including those customized to the software requirements space, can simply be "plugged in" to our process to improve test selection results.

2.3 Agile Regression Testing

With agile methodologies gaining widespread adoption, the demand for rapid, iterative software testing has increased, shifting away from traditional models such as waterfall [22]. The rapid iterative nature of agile methods, combined with an emphasis on automated testing and pipeline delivery, suggests there simply isn't time to correct defects found during the testing process. Gone are "test-and-fix" *phases* on large software versions (large meaning "lots of new features and fixes"), replaced by automated pipelines that pass or fail smaller software *increments* more often. For this reason we emphasize RTS and not RTP in our research as it reflects modern practice.

Kandil et al. [2017] [12] proposed clustering similar test cases to select regression tests, showing improved fault detection, though limitations like potential bias in selecting representative test cases remain. Ali et al. [2020] [1] employed code coverage and change impact analysis for test selection, while Koivuniemi [2017] [14] explored RTS based on code coverage and dependencies, leading

to reduced feedback times without compromising test robustness. Elbaum et al. [2014] [10] discussed continuous integration (CI) specific challenges in optimizing regression testing, showing cost-effective improvements in large-scale projects. Shi et al. [2019] [20] contrasted module- and class-level RTS, finding that while module-level RTS selects more test cases with minimal overhead, class-level RTS is more selective but time-intensive. Their study using Travis CI on open-source Java projects showed RTS reduced testing time compared to running all tests, though cloud-based overhead remains. Yu and Wang [2018] [24] noted that while RTS may be redundant for infrequent commits, it becomes crucial for rapid changes, as frequent commits often affect a small portion of the source code. For most Java projects, class-level RTS proved more efficient than method-level RTS or exhaustive testing. These advances in regression testing, from code coverage to machine learning integration, highlight the need for continuous improvements as CI becomes more prevalent.

Various regression test selection approaches, including requirements-based, textual similarity-based, and agile-specific techniques are invaluable in upholding software quality. Agile methodologies, which underscore the importance of delivering incremental business value to stakeholders, serve as a reassuring reminder of the effectiveness of these methodologies. 'Value' is the cornerstone of software success. Regression testing methods play a crucial role in swiftly identifying multiple defects; however, a defect indicates that the software is not in the 'done' state, thereby diminishing its value. Therefore, in agile environments, regression testing must also ensure that changes do not disrupt any business requirements, technical accuracy, and the preservation of value post-change. Sustaining the business value and technical integrity of software is a pivotal aspect of agile development. Nevertheless, the existing literature on agile regression testing has yet to integrate business value as a metric for evaluating regression testing. As such, our motivation for this paper is to address the research question:

RQ: How does value-based requirement-driven regression test selection compare to random selection in the context of agile?

3 RTS Algorithm

This section presents our design rationale and algorithm for RTS based on a set of agile factors. We start by giving an agile primer to ensure agile-specific language is consistently interpreted, and then present the algorithm.

3.1 An Agile Primer

We presume the reader has a basic understanding of agile software development, but to ensure the language used in describing our approach is consistently understood, we summarize these concepts briefly here. First, we use the term *agile software development* to refer to methodology not a specific process model; agile methods emphasize incremental and iterative development. For this research it is not important which specific process model is used, though it may be helpful to the reader to assume a *Scrum*-like model [?]. Consistent with Scrum, we assume requirements are written as *user stories*, though this is not strictly necessary. The more important piece is that user stories have 1) a defined acceptance test, and 2) an associated *business value* assigned by a *product owner*. The

acceptance tests are candidate regression tests as they are based on observable behaviors of the application; we do not consider unit tests as these (whether black-box or white-box) are based on the implemented code. In practice business values are *interval scale* measures used by Scrum teams to prioritize feature delivery. Different lightweight estimation techniques exist, such as *planning poker*, which apply convergence-based estimating concepts using interval scales such as a Fibonacci sequence or "T-shirt sizing". For our research we assume Fibonacci numbers. Iterations are commonly referred to as *sprints*, which originally in practice were suggested to be roughly one month [?] but in modern practice are much shorter (1-2 weeks). Some organizations reduce sprints to a single day or even a single change event (e.g. feature completion leading to continuous deployment). The Scrum team collaboratively determines which user stories to implement in a new sprint through a planning process, and completes a sprint with a *review* with the Product Owner of all stories passing acceptance tests. From our research perspective the length of a sprint impacts the amount of time available for regression test execution, but otherwise is inconsequential.

3.2 Algorithm Design Factors

Our approach to RTS incorporates the following factors:

- business value of the current user stories
- similarity of current user stories to prior (already delivered) requirements
- the business value of such user stories (from the previous bullet)
- the available time for regression test execution and the time of execution of each test

The rationale for applying these factors is as follows. In agile processes we seek to maximize the value delivered. This factor prioritizes the work an agile team does on a daily basis, and therefore should influence the tests that are performed. The first bullet is self-evident as business value is the primary way to prioritize current requirements for agile teams. From this prioritized list we try to identify those requirements that are most like these requirements. By doing so, we identify a set of already implemented requirements whose features are more likely to be adversely impacted by the new software version. Incorporating the business value of those already delivered user stories gives us an indication of risk - if a new feature breaks existing features, just how critical or valuable were those existing features? Note that anecdotally, this might be an analysis function performed by a quality assurance team or the development team itself (criticality of a defect). But we are not concerned with the time required to fix a defect, we are concerned whether existing value inherent in the software is negatively impacted. Finally, the time available to test and the time needed for each test is obviously important, as these define the inherent constraints of the RTS problem in an agile environment.

3.3 Regression Test Selection Implementation

The RTS algorithm, as shown in algorithm 1, combines textual similarity, business value, and test execution time to guide the selection of test cases. The first step is to create a matrix of current and prior user stories, with each cell storing a textual similarity

score. This gives us a strength-oriented relationship between each pair of stories, current and prior. Then for each row in the matrix (the SimilarityList), we compute two additional values; a sum of the business values of each story pair, and the summed execution time of all test cases associated with the prior story. Recall user stories have one or more acceptance tests, so the union of all acceptance tests from all prior (already delivered) user stories are the candidate set for regression test selection. This substep simply determines how long it would take to run all tests associated with a prior acceptance test. To complete step 2 we compute what we refer to as the *Importance Value* for each prior user story as the product of similarity and value, normalized by the execution time of all that user stories' acceptance test cases. Effectively we determine a per unit time of *value preserved* from previous sprints using this ratio. The final step, step 3, is simply a greedy algorithm based on a sorted list of Importance Values. The intelligence of this algorithm comes from the combination of similarity and value as the guiding heuristics. The algorithm seeks to protect as much delivered value as possible, which is in stark contrast to typical RTS approaches that seek to maximize defects discovered.

Our implementation was done in Python, with the spaCy Python library used to calculate the Similarity Index (SI), comparing the text of user stories from current releases with those from prior releases. This similarity metric helped identify the user stories most relevant to current changes. As mentioned above, spaCy is an OTS library for text similarity, and suitable library or algorithm could be substituted to improve results, though this was not a focus of our research at this stage. Business value (BV) prioritized user stories important to the business, ensuring that test cases linked to high-value functionalities were prioritized.

4 Methods

This section details the methodology for implementing and evaluating the value-preserving regression test selection (RTS) method. The approach focuses on selecting test cases based on textual similarity between user stories across releases and business value to ensure higher valued functionalities are prioritized during regression testing.

In a recent comprehensive systematic literature review, Pan et al. [2022] [17] meticulously surveyed the landscape of regression testing for agile methods. Their analysis revealed that only half of the 26 scrutinized studies made their datasets accessible. These selected studies were evaluated against six metrics and subsequently categorized as repeatable or reproducible. Furthermore, the review identified that 21 of the evaluated studies sourced their data from publicly accessible datasets, notably including the Software-artifact Infrastructure Repository (SIR). However, the data primarily compiled test cases from code modifications before 2016, which may not adequately represent the attributes and metrics of agile methodologies today. This limitation highlights the urgent need to develop up-to-date datasets incorporating agile-centric features to facilitate a thorough evaluation of value-driven approaches in test selection for agile environments. The remainder of the section presents the data preparation, the RTS algorithm, and the evaluation process.

Algorithm 1 Value-Preserving Regression Test Selection

```

1: Input: CurrentReleaseStories[], PriorReleaseStories[], BusinessValue[], ExecutionWindow
2: Output: TotalBusinessValue
3: Initialize SelectedStories[]
4: Set RemainingExecutionTime to ExecutionWindow
5: Set TotalBusinessValue to 0
6: Initialize ImportanceValue[]
7: Step 1: Text Similarity Calculation
8: for each story  $s_i$  in CurrentReleaseStories[] do
9:   for each story  $s_j$  in PriorReleaseStories[] do
10:    Calculate SI between  $s_i$  and  $s_j$  using spaCy
11:    Store the similarity score,  $s_i$ , and  $s_j$  in SimilarityList[]
12:   end for
13: end for
14: Step 2: Importance Value Calculation
15: for each pair in SimilarityList[] do
16:   Get the business values  $BV_i$  and  $BV_j$ 
17:   Compute  $BV_{sum} = BV_i + BV_j$ 
18:   Calculate total execution time for all test cases associated with  $s_j$ :  $ET_{total}$ 
19:   Compute Importance Value (IV) as:

$$ImportanceValue[] = \frac{SI \times BV_{sum}}{ET_{total}}$$

20: end for
21: Step 3: Select Prior Release Stories within Execution Window
22: for each prior release story  $s_j$  in PriorReleaseStories[] (sorted by descending ImportanceValue[]) do
23:   while  $ExecutionTime - ET_{total} \geq 0$  do
24:    Add  $s_j$  to SelectedStories[]
25:    Add  $BV_j$  to TotalBusinessValue
26:    Subtract  $ET_{total}$  from ExecutionTime
27:   end while
28: end for
29: Step 4: Output Total Business Value
30: Return TotalBusinessValue
=0

```

4.1 Data Preparation

Simulations were conducted by synthetically adding attributes to the extracted user stories from the TAWOS dataset [21], which contains user stories written in natural language. Then, user stories were categorized into current and prior releases. Business values were assigned between 1-89 (Fibonacci series) to each user story [5], simulating real-world conditions with varying business priorities.

Synthetic test cases were associated with the user stories, and execution times followed a right-skewed distribution, reflecting the typical variation in test case execution times. The skewness is purposefully kept toward the right, considering that higher execution time test cases are less in number.

The total time available for regression testing, referred to as the execution window, was varied across simulations, simulating different time-constrained testing scenarios in agile environments. Table 1 below shows the variables used in the simulation.

Table 1: Key variables used in the simulations

Variable	Description
Business Value	Values ranging from 1-89
User Story(s)/Release	Count of user stories/release
Execution Time	Execution time of each test case
Execution Window	Available time for regression testing

4.2 Performance Measure

The RTS approach's performance was evaluated against two other approaches. The primary metric for evaluating the methods was the total business value preserved, calculated by summing the business values of the user stories whose test cases were chosen for regression. The results were analyzed to assess how effectively the RTS method preserved business value compared to the two baseline methods, given the constraints of the available execution window.

The first approach was the Random Selection Process (RSP), which randomly selected test cases within the available time without considering textual similarity or business value. This comparison sought to establish whether the value-based RTS method conferred a substantial advantage over random selection.

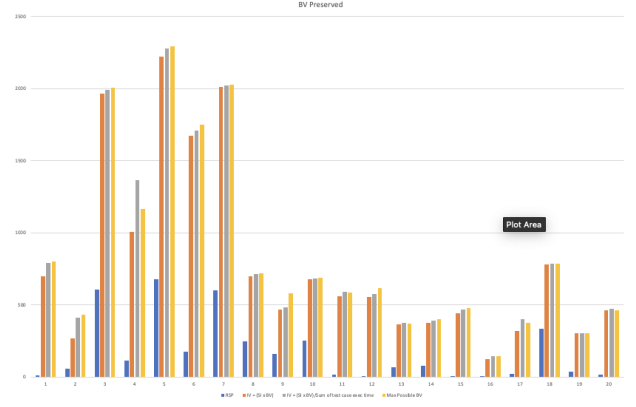
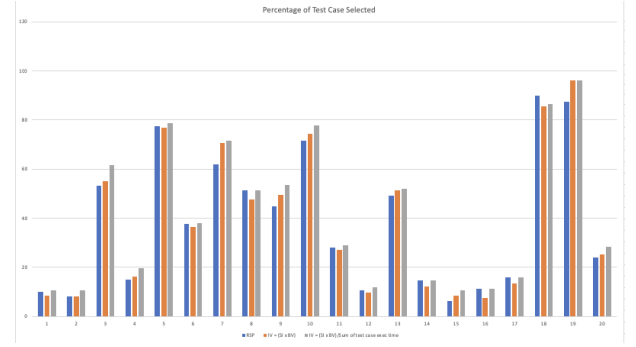
The second baseline, IV ($SI \times BV$), selected test cases based on the importance value calculated as the product of similarity index (SI) and business value (BV). This approach did not consider execution times, allowing us to isolate the impact of incorporating test case execution time into the selection process.

5 Results

This section presents the results of the RTS approach using two importance value formulations, comparing them with the random selection process discussed in the previous section. We focus on the preserved value relative to the maximum preserved value, which is the total business value of all user stories determined by the selection process. We utilized two variations to compute importance values. It is important to remember that we are conducting RTS, not regression test prioritization. Even though our approach could facilitate prioritization, we are not considering it, as we need to consider defect severity and time for fixing and retesting. Therefore, if a regression test fails, the corresponding change causing the failure should be excluded from the upcoming increment. To illustrate that

Figure 1 shows variability across different simulation factors, a bar chart of 20 simulation runs is presented in Figure 1 [Color codes:- Blue: RSP, Orange: $IV = (SI \times BV)$, Grey: $IV = (SI \times BV)/\text{Sum of test case exec time}$, Yellow: Max Possible BV]. These grouped bar charts display the total business value available in previous releases and the maximum possible value preserved. Figure 2 shows the percentage of test cases selected for each of the approach [Color codes:- Blue: RSP, Orange: $IV = (SI \times BV)$, Grey: $IV = (SI \times BV)/\text{Sum of test case exec time}$]

The results of the 20 simulations, presented in Figures 1 and 2, demonstrate apparent differences in the effectiveness of the three test case selection methods—Random Selection Process (RSP), $IV = SI \times BV$, and $IV = SI \times BV / \text{Sum of Test Case Execution Time}$ —in terms of both business value preserved and the percentage of test

**Figure 1: BV Preserved****Figure 2: Percent of TC selected**

cases selected. These differences are strongly influenced by the available time for regression testing in each simulation.

In terms of business value preserved, as illustrated in Figure 1, the RSP method consistently under-performs across all simulations. The randomness inherent in this method results in an inefficient selection of test cases, leading to the lowest amount of business value preserved in nearly every instance. For example, in simulations 4, 5, and 6, where the available time is relatively high, RSP preserves significantly less business value than the other methods. This trend is consistent across both scenarios with abundant and constrained execution time, highlighting the fundamental inefficiency of RSP when business value is not a guiding factor in the selection process.

In contrast, the $IV = SI \times BV$ method, which prioritizes test cases based on their business value and similarity to prior release user stories, performs significantly better in preserving business value, especially when the execution time is not a limiting factor. In simulations with ample available time, such as runs 5 and 7, this method preserves business value close to the Maximum Possible BV, demonstrating its effectiveness in selecting high-value test cases when time constraints are relaxed. However, in simulations with more restricted time, such as runs 10 and 12, this method shows a noticeable decline in performance. The reduction in business value preserved in these scenarios suggests that while this approach effectively

prioritizes high-value test cases, it fails to account for execution time, making it less efficient when time is a critical constraint.

The $IV = SI \times BV / \text{Sum of Test Case Execution Time}$ method, on the other hand, demonstrates consistent performance across both time-rich and time-constrained simulations. By incorporating execution time into the Importance Value calculation, this method balances preserving business value and adhering to the available execution window. In simulations with limited time, such as runs 10 and 12, this approach outperforms RSP and $IV = SI \times BV$ by selecting test cases that maximize business value while fitting within the restricted time frame. Even in simulations with more available time, such as runs 5 and 9, the $IV = SI \times BV / \text{Execution Time}$ method remains competitive, preserving business value close to the maximum possible. This indicates that including execution time in the selection process provides a significant advantage, particularly in time-constrained environments.

The percentage of test cases selected, as shown in Figure 2, further illustrates the adaptability of each method to varying time constraints. The RSP method, due to its random nature, exhibits erratic behavior, with the percentage of test cases selected varying widely from simulation to simulation. This inconsistency reflects the lack of strategic prioritization in the RSP approach, resulting in inefficient test case selection regardless of the available time. In contrast, the $IV = SI \times BV$ method selects a higher percentage of test cases when more time is available, as seen in simulations such as runs 5 and 7, where it selects more than 70% of the test cases. However, in simulations with tighter time constraints, such as runs 10 and 12, the percentage of selected test cases drops significantly, illustrating the method's difficulty in optimizing under limited execution time.

The $IV = SI \times BV / \text{Sum of Test Case Execution Time}$ method consistently selects a moderate to high percentage of test cases, regardless of the available time. Even in simulations with severe time constraints, this method can select many test cases while ensuring they fit within the execution window. For instance, in runs 10 and 12, where the available time is minimal, this method selects a higher percentage of test cases than the other two, reflecting its efficiency in balancing business value and time constraints.

The influence of the execution time on the performance of these methods is evident across all simulations. In scenarios with ample time for regression testing, such as simulations 5, 9, and 19, both IV-based methods perform similarly, as the larger execution window allows for more comprehensive test case selection. However, in simulations with tighter execution windows, such as runs 10, 12, and 15, the differences between the methods become more pronounced. The $IV = SI \times BV$ method, which does not factor in execution time, struggles to maintain high performance in these scenarios. In contrast, the $IV = SI \times BV / \text{Execution Time}$ method consistently outperforms the other approaches by selecting the most valuable test cases that can be executed within the limited time.

In summary, the results show that while $IV = SI \times BV$ is highly effective when execution time is abundant, it becomes less efficient under time constraints. The $IV = SI \times BV / \text{Sum of Test Case Execution Time}$ method, which accounts for execution time in the selection process, is more adaptable and efficient across time-rich and time-constrained scenarios. The RSP method, on the other hand, consistently performs poorly, demonstrating the inefficiency

of random selection when business value and execution time are not considered.

6 Discussion

This section answers the research question: *How does value-based, requirement-driven regression test selection compare to random selection in the context of agile?*

By analyzing two Importance Value (IV) formulations— $IV = SI \times BV$ and $IV = SI \times BV / \text{Execution Time}$ —, we examined how business value (BV) and execution time influenced the selection of test cases across 20 simulations. By focusing on both business value preservation and the efficiency of test execution, the RTS algorithms aimed to improve the regression testing process in agile environments. The results across 12 analyzed simulations offer a clear picture of the strategies employed by each IV formulation, and these insights can be generalized to the remaining eight simulations, highlighting the practical trade-offs made by the algorithms in selecting test cases under varying time constraints.

The selection of user stories and test cases across the 12 analyzed simulations reflects the decision-making of the regression test selection (RTS) algorithms based on two Importance Value (IV) formulations: $IV = SI \times BV$ and $IV = SI \times BV / \text{Execution Time}$. Both formulations aim to maximize business value (BV) during regression testing but with differing priorities. Analyzing these 12 simulations provides insight that can be generalized to the remaining eight simulations due to the consistent pattern of selections observed.

In the $IV = SI \times BV / \text{Execution Time}$ formulation, execution time plays a crucial role in the algorithm's selection of test cases. This approach prioritizes test cases with the best balance between business value and execution time, ensuring that the most valuable test cases are selected within the constraints of the available testing window. For example, in simulations such as 1 and 2, many test cases with moderate business value but relatively low execution times were selected, allowing the RTS algorithm to efficiently use the available time while preserving as much business value as possible. This pattern is particularly evident in simulation 3, where test cases like TC186 and TC193 were chosen not because they had the highest business value but because their shorter execution times allowed them to fit within the limited testing window. This selection strategy enables the algorithm to prioritize the overall business impact of the regression testing process without exceeding time limits.

Execution time significantly influences the test case selection process under the $IV = SI \times BV / \text{Execution Time}$. In cases where available time is limited, the algorithm leans towards test cases with a favorable business value-to-execution time ratio. This ensures that as many valuable test cases as possible can be run within the available testing window, maximizing business value retention while adhering to time constraints. The selections in simulations 3 and 4 demonstrate this, where test cases with moderate business value but shorter execution times were preferred, ensuring that the testing process could proceed efficiently. By contrast, if execution time had not been considered, longer-running test cases might have been chosen, limiting the number of test cases that could be executed and thus reducing overall business value preservation.

On the other hand, the $IV = SI \times BV$ formulation focuses solely on business value and the similarity index (SI) between user stories without factoring in execution time. While this approach would prioritize test cases that maximize business value, it risks overloading the testing process with long-running test cases, especially in time-constrained scenarios. In such cases, the lack of consideration for execution time would lead to inefficient use of the available testing window. This explains why, in the analyzed simulations, no rows were populated under this formulation—indicating that time constraints significantly guided test case selection in these scenarios.

The observed selections across the 12 simulations demonstrate the effectiveness of $IV = SI \times BV / \text{Execution Time}$ in balancing business value preservation with execution efficiency. The consistent selection of test cases that optimize business value and time reflects the algorithm's ability to make practical trade-offs, ensuring that the testing window is used as efficiently as possible. This approach proves particularly advantageous when execution time is limited, as seen in simulation 5, where shorter test cases with lower business value were selected to ensure that the available time was fully utilized.

We can generalize these findings by examining these 12 simulations to the remaining eight. The selection patterns remain consistent: in time-constrained environments, the algorithm favors test cases with a favorable business value-to-execution time ratio, maximizing the number of test cases that can be executed while preserving essential business functionality. This strategy results in a more comprehensive and efficient regression testing process, particularly under time restrictions, and ensures that the business impact of regression testing is maximized without exceeding time limits.

In conclusion, the inclusion of execution time in the $IV = SI \times BV / \text{Execution Time}$ formulation significantly impacts the selection of test cases. This formulation allows the algorithm to prioritize test cases with the highest overall benefit, optimizing for business value and execution time. In contrast, the $IV = SI \times BV$ formulation would prioritize high-value test cases but may lead to inefficiencies in time-constrained scenarios. The RTS algorithm's adaptability in selecting test cases with lower execution times, especially in scenarios with restricted time, ensures that regression testing remains efficient and effective. This analysis of 12 simulations confirms that these patterns can be extended to the rest, making the algorithm a valuable tool for maintaining regression quality under varying time constraints.

7 Limitations

This preliminary study is based on simulated scenarios with certain limitations. We made key assumptions using Fibonacci numbers to assign business value and the right-skewed distribution of execution times, which were not based on empirical data but on widely accepted practices and theoretical considerations. While these choices help model a practical regression test selection environment, they may only partially represent real-world dynamics. Other simulation factors, such as the specific characteristics of test cases and execution windows, were derived from the authors' interpretations and understanding of typical agile development processes. These

factors are open to different interpretations and may vary in real-world settings. For instance, we did not consider factors like setup and tear-down times for test cases, which are common in practice and could influence test case selection. Future research would benefit from validating these assumptions and extending the scope to include more comprehensive, real-world data to strengthen the practical applicability of the proposed method.

8 Conclusions and Future Work

This study introduced a value-driven approach to regression test case selection, offering a more strategic method for optimizing the regression testing process. Focusing on business value allows teams to prioritize user stories that deliver the most significant impact, even when testing resources are limited. The comparison with random selection methods confirmed that this value-based strategy could ensure the preservation of critical functionality with fewer test cases, making it a robust alternative for regression testing in agile environments. The method's ability to balance preserving business value and optimizing test execution opens the door for more efficient and targeted regression suites, contributing to higher software quality and more efficient use of testing resources.

However, this work inspires several fascinating possibilities for further refinement and enhancement. One promising avenue for future research involves expanding the method's consideration of time factors. While the current approach focuses solely on test case execution time, real-world testing environments often involve additional overheads, such as setup and tear-down times. These factors can significantly impact the overall efficiency of the regression testing process and warrant inclusion in future iterations of the method. By accounting for these additional time constraints, future work could offer a more precise and holistic approach to test case selection, further enhancing the algorithm's utility in real-world applications.

Moreover, there is considerable potential in exploring the partial selection of test cases within user stories. In this study, we approached each user story holistically, selecting all associated test cases based on business value. However, introducing the ability to prioritize and select only the most critical test cases within a user story could provide a more flexible and granular approach. This partial selection would allow teams to test partial business value when resources are constrained, optimizing the regression test suite for agile environments where time and resources are scarce.

In summary, the proposed value-driven RTS approach has demonstrated its potential to improve regression testing by focusing on business value. Future work can build upon this foundation by incorporating additional time factors, enabling partial test case selection, and leveraging historical test case data. Together, these enhancements promise to refine the approach and bring it closer to practical, real-world application, where the demands of agile development necessitate both efficiency and precision in testing strategies. This potential should inspire and motivate further exploration of the value-driven approach in the field of software testing and agile development.

References

- [1] S. Ali, Y. Hafeez, S. Hussain, and S. Yang. 2020. Enhanced regression testing technique for agile software development and continuous integration strategies.

- Software Quality Journal* 28 (2020), 397–423. doi:10.1007/s11219-019-09492-2
- [2] Agile Alliance. 2001. *Manifesto for Agile Software Development*. <https://agilemanifesto.org>. Retrieved September 12, 2024.
- [3] M. J. Arafeen and H. Do. 2013. Test case prioritization using requirements-based clustering. In *Proceedings of the 2013 IEEE 6th International Conference on Software Testing, Verification and Validation*. IEEE, 312–321.
- [4] M. Azizi and H. Do. 2018. Retest: A cost-effective test case selection technique for modern software development. In *Proceedings of the 2018 IEEE 29th International Symposium on Software Reliability Engineering (ISSRE)*. IEEE, 144–154.
- [5] C. Chauhan, M. Dakoliya, and R. K. Sharma. 2023. Use of Fibonacci Sequence in Project Estimation. *Indian Journal of Science and Technology* 16, 33 (2023), 2649–2652.
- [6] P. K. Chittimalli and M. J. Harrold. 2008. Regression test selection on system requirements. In *Proceedings of the 1st India Software Engineering Conference*. ACM, 87–96.
- [7] CrowdStrike. 2024. *Channel File 291 Incident Root Cause Analysis*. Technical Report. CrowdStrike, Inc., Austin, TX. <https://www.crowdstrike.com/wp-content/uploads/2024/08/Channel-File-291-Incident-Root-Cause-Analysis-08.06.2024.pdf>. Retrieved from CrowdStrike website.
- [8] CrowdStrike. 2024. *What is CI/CD?* <https://www.crowdstrike.com/cybersecurity-101/security-it-operations/what-is-ci-cd/>. Retrieved September 12, 2024.
- [9] S. Das and K. Gary. 2021. Agile transformation at scale: A tertiary study. In *Agile Processes in Software Engineering and Extreme Programming—Workshops: XP 2021 Workshops, Virtual Event, June 14–18, 2021, Revised Selected Papers*, Vol. 22. Springer International Publishing, 3–11.
- [10] S. Elbaum, G. Rothermel, and J. Penix. 2014. Techniques for improving regression testing in continuous integration development environments. In *Proceedings of the 22nd ACM SIGSOFT International Symposium on Foundations of Software Engineering*. ACM, New York, NY, USA, 235–245.
- [11] Jeanette Heidenberg, Max Weijola, Kirsi Mikkonen, and Ivan Porres. 2012. A model for business value in large-scale agile and lean software development. In *Systems, Software and Services Process Improvement: 19th European Conference, EuroSPI 2012, Vienna, Austria, June 25–27, 2012. Proceedings*, Vol. 19. Springer Berlin Heidelberg, Berlin, Germany, 49–60.
- [12] P. Kandil, S. Moussa, and N. Badr. 2017. Cluster-based test cases prioritization and selection technique for agile regression testing. *Journal of Software: Evolution and Process* 29, 6 (2017), e1794. doi:10.1002/smr.1794
- [13] Z. Khalid and U. Qamar. 2019. Weight and cluster based test case prioritization technique. In *Proceedings of the 2019 IEEE 10th Annual Information Technology, Electronics and Mobile Communication Conference (IEMCON)*. IEEE, 1013–1022.
- [14] J. Koivuniemi. 2017. *Shortening feedback time in continuous integration environment in large-scale embedded software development with test selection*. Master's thesis. Master's Thesis, J. Koivuniemi.
- [15] S. Nidhra and J. Dondeti. 2012. Black Box and White Box Testing Techniques: A Literature Review. *International Journal of Embedded Systems and Applications (IJESA)* 2, 2 (2012), 29–50.
- [16] Institute of Project Management. 2022. *16th Annual State of Agile Report*. <https://instituteofprojectmanagement.com/wp-content/uploads/2023/01/AR-SA-2022-16th-Annual-State-Of-Agile-Report.pdf>. Retrieved September 12, 2024.
- [17] R. Pan, M. Bagherzadeh, T. A. Ghaleb, and L. Briand. 2022. Test case selection and prioritization using machine learning: a systematic literature review. *Empirical Software Engineering* 27, 2 (2022), 29. doi:10.1007/s10664-021-10046-1
- [18] Mary Poppendieck. 2011. Principles of Lean Thinking. *IT Management Select* 18 (2011), 1–7.
- [19] Gregg Rothermel and Mary Jean Harrold. 1997. A safe, efficient regression test selection technique. *ACM Transactions on Software Engineering and Methodology (TOSEM)* 6, 2 (1997), 173–210.
- [20] A. Shi, P. Zhao, and D. Marinov. 2019. Understanding and improving regression test selection in continuous integration. In *Proceedings of the 2019 IEEE 30th International Symposium on Software Reliability Engineering (ISSRE)*. IEEE, New York, NY, USA, 228–238.
- [21] SOLAR-group. n.d.. *GitHub - SOLAR-group/TAWOS: The Tawosi Agile Webhosted Open-Source Issues dataset*. <https://github.com/SOLAR-group/TAWOS>. Retrieved September 12, 2024.
- [22] Helge Spieker, Arnaud Gotlieb, Dusica Marijan, and Morten Mossige. 2017. Reinforcement learning for automatic test case prioritization and selection in continuous integration. In *Proceedings of the 26th ACM SIGSOFT International Symposium on Software Testing and Analysis*. ACM, New York, NY, 12–22.
- [23] P. R. Srivastva, K. Kumar, and G. Raghurama. 2008. Test case prioritization based on requirements and risk factors. *ACM SIGSOFT Software Engineering Notes* 33, 4 (2008), 1–5.
- [24] T. Yu and T. Wang. 2018. A study of regression test selection in continuous integration environments. In *Proceedings of the 2018 IEEE 29th International Symposium on Software Reliability Engineering (ISSRE)*. IEEE, 135–143.